

1. Validar el pedido de pizzas

Segunda_Pag/formulario.js | Líneas 3-43

El DOM representa el HTML como objetos que JavaScript puede consultar. Este archivo localiza los controles y comprueba el pedido antes de enviarlo. El script está al final del body: los elementos ya existen cuando se ejecuta.

Referencias a los controles | líneas 3-9

Las siete constantes guardan el formulario, las casillas nap, jq y muz, y sus cantidades cant_nap, cant_jq y cant_muz. document.getElementById busca cada elemento por su id. const impide reasignar la referencia, pero el control puede cambiar. checked devuelve true o false; value devuelve texto, incluso en un input de tipo number.

Evento y primera condición | líneas 13-19

```
formulario.addEventListener("submit", function(event){
  if (!nap.checked && !jq.checked && !muz.checked){
    event.preventDefault();
    alert("Debes seleccionar al menos una opcion para continuar")
    return;
  }
  // Después se comprueban las tres cantidades.
});
```

addEventListener registra un callback: una función que el navegador ejecutará cuando ocurra submit. event representa ese evento. ! niega cada estado y && exige que los tres sean verdaderos: la condición detecta que todas las casillas están desmarcadas. Se permite elegir una, dos o tres variedades.

preventDefault() cancela el envío; alert informa al usuario; return termina el callback para evitar más comprobaciones. Son acciones diferentes: return por sí solo no cancela un envío con addEventListener.

Tres reglas de cantidad | líneas 22-38

```
if (nap.checked && Number(cant_nap.value) < 1){
  event.preventDefault();
  alert("Ingrese una cantidad correcta");
  return;
}
```

El mismo bloque se repite con jq/cant_jq y muz/cant_muz. Si una pizza está marcada y su cantidad es menor que uno, se bloquea. Number convierte el texto: Number("2") es 2 y Number("") es 0. && tiene cortocircuito: si la casilla no está marcada, omite la segunda expresión.

El navegador valida antes de submit: required exige nombre y dirección; min="1" y el paso numérico pueden bloquear 0 o 1.5 antes del alert. Si no se cancela, el form hace GET a recibo.html. El comentario de la línea 21 está invertido: menor que uno es el error.

Los fragmentos reúnen el código clave; el comentario abreviado del primer bloque es explicativo. HTML relacionado: form_pizza.html:14,18,23,40-91,117.

2. Construir el recibo

Segunda_Pag/recibo.js | Líneas 1-22

De los campos a la URL

El formulario envía pares name=value mediante GET. id identifica el elemento en el DOM; name será la clave enviada; value es su contenido. La casilla de napolitana tiene id="check-nap", name="tipo-pizza-nap" y value="napolitana". Una casilla desmarcada no se envía, aunque su campo de cantidad independiente sí puede enviarse.

Leer los parámetros | líneas 1-5

```
const urlParams = new URLSearchParams(window.location.search);
const nombre = urlParams.get('usuario');
const direccion = urlParams.get('direccion');
const telefono = urlParams.get('telefono');
const pedido = [];
```

window.location.search obtiene la consulta de la URL, desde ?. new crea un objeto URLSearchParams para interpretarla y decodificarla. get obtiene una cadena por su clave, o null si no existe. Una clave presente pero vacía devuelve "". pedido empieza como arreglo vacío. Aquí no se convierten cantidades a número porque solo se presentan como texto.

Mostrar el cliente | líneas 7-9

```
document.getElementById('nombre').textContent += nombre;
```

Las otras dos líneas hacen lo mismo con dirección y teléfono. textContent escribe texto literal, sin interpretar HTML. += concatena el contenido previo con el dato: "Nombre: " más "Ana" produce "Nombre: Ana". Con = se perdería la etiqueta. Abrir el recibo sin parámetros muestra null en los datos del cliente.

Agregar variedades y unirlos | líneas 12-22

```
if (urlParams.get('tipo-pizza-nap') === 'napolitana') {
  pedido.push(`Napolitana: ${urlParams.get('cantidad-pizza-nap')}`);
}
document.getElementById('pedido').textContent += pedido.join(', ');
```

Hay otros dos if equivalentes para jamon-queso y muzzarela. === compara sin convertir tipos. Son if independientes porque el pedido admite varias variedades; una cadena else if solo tomaría la primera rama verdadera.

push agrega una cadena al arreglo. Las comillas invertidas crean una plantilla y \${...} inserta la cantidad consultada. const permite modificar el arreglo, aunque impide reasignarlo. join devuelve una cadena con coma y espacio entre elementos, sin cambiar el arreglo. La última línea la añade al párrafo Pedido:.

Ejemplo: dos napolitanas y una muzzarela generan ["Napolitana: 2", "Muzzarela: 1"] y luego "Napolitana: 2, Muzzarela: 1". No se muestran tamaños ni se calculan precios. La URL puede alterarse y el recibo no vuelve a validar cantidades.

recibo.js se ejecuta al cargar la página; no registra eventos. Los párrafos están en recibo.html:15-18 y el script se carga en la línea 25.

3. Elegir país y consultar sus datos

Proyectos_Medios_Pago/script.js | Líneas 1-44

Un catálogo local | líneas 1-31

select_paises es un objeto con seis claves: CO, MX, AR, CL, ES y PE, para Colombia, México, Argentina, Chile, España y Perú. Cada país contiene otro objeto con dos arreglos de cinco cadenas: ciudad y departamento. Por ejemplo, CO.ciudad contiene Bogotá, Medellín, Cali, Barranquilla y Cartagena.

Las llaves definen objetos; los dos puntos relacionan una clave con su valor; los corchetes definen arreglos y las comas separan elementos o propiedades. Es un objeto escrito en JavaScript: no se consulta una API, no se usa JSON.parse y no hay base de datos.

Datos frente a elementos del DOM | líneas 33-35

select_paises, en plural, es el catálogo. select_pais es el control con id="pais"; select_ciudad y select_departamento son los controles con id="ciudad" e id="departamento". Las tres referencias al DOM se obtienen con getElementById.

Leer y limpiar cuando cambia el país | líneas 37-41

```
select_pais.addEventListener("change", function(){
  const select_v = select_pais.value;
  select_ciudad.innerHTML = `<option value=""> Elige una ciudad </option>`;
  select_departamento.innerHTML = `<option value=""> Elige un departamento </option>`;
  // A continuación se consulta el catálogo y se llenan las listas.
});
```

change ejecuta el callback cuando el usuario cambia la selección. Si elige Colombia, value devuelve "CO", no el texto visible "Colombia". select_v guarda esa clave. Aquí no se declara event porque no se necesita consultar ni cancelar el evento.

innerHTML interpreta la cadena como HTML y crea una opción. Usar = reemplaza las opciones anteriores por un mensaje guía con value vacío. Esto elimina selecciones del país anterior y evita mezclar listas. La limpieza ocurre incluso si después no se encuentra un país válido.

Acceso dinámico y protección | líneas 43-44

```
if (select_paises[select_v]) {
  const info = select_paises[select_v];
  // Los dos forEach utilizan las listas de info.
}
```

Los corchetes usan el contenido de select_v como clave: con "CO" obtienen el objeto de Colombia. Es distinto de .select_v, que buscaría una propiedad llamada literalmente select_v. El objeto encontrado se interpreta como verdadero; la clave vacía devuelve undefined y el if no entra. info guarda una referencia al país, no una copia.

Para agregar otro país: añade una opción al HTML y una clave idéntica en el objeto, con ambos arreglos. La lógica del evento y los recorridos se reutiliza.

registro.html:88-104 define los selectores; línea 139 carga el script. Los comentarios de abreviación en los fragmentos son explicativos.

4. Llenar las listas con forEach

Proyectos_Medios_Pago/script.js | Líneas 46-54

```
info.departamento.forEach(function(departamento) {  
    select_departamento.innerHTML +=  
        `<option value="${departamento}">${departamento}</option>`;   
});  
  
info.ciudad.forEach(function(ciudad) {  
    select_ciudad.innerHTML +=  
        `<option value="${ciudad}">${ciudad}</option>`;   
});
```

Qué pasa en cada vuelta

forEach ejecuta su callback una vez por elemento del arreglo. En el primer recorrido, departamento es una cadena como "Antioquia". No es el arreglo info.departamento ni el control select_departamento. El segundo recorrido hace lo mismo con cada ciudad.

La plantilla inserta el nombre dos veces: en value, que será el dato de la opción, y entre las etiquetas, que será el texto visible. Por ejemplo, crea <option value="Antioquia">Antioquia</option>.

+= conserva el HTML existente y agrega la siguiente opción. Si se usara = dentro del recorrido, cada vuelta borraría la anterior y solo quedaría la última. Primero terminan los departamentos y luego se recorren las ciudades: son operaciones síncronas con datos locales. El retorno de forEach es undefined; aquí interesa su efecto sobre el DOM.

Sigue el flujo en un ejemplo

Al abrir registro.html, las dos listas están vacías porque todavía no ocurrió change. Elegir Colombia pone "CO" en select_v: se limpia, se obtiene info y se agregan cinco nombres más una guía por lista. Cambiar a México quita lo anterior y carga MX. Volver al país vacío limpia, pero el if no entra: queda solo una guía en cada lista.

Tres detalles que pueden preguntarte

1. Ciudad y departamento son independientes. Ambos dependen del país, pero elegir un departamento no filtra ciudades. No existe un listener para ese control; por eso se puede elegir Antioquia y Cali a la vez.
2. Borrar usa reset de HTML. Restablece valores, pero no elimina las opciones creadas ni dispara automáticamente el change del país. Pueden quedar opciones del país anterior. Habría que sincronizar explícitamente las listas con el reinicio.
3. innerHTML += reinterpreta HTML en cada vuelta. Funciona con estas listas pequeñas y fijas. Una mejora sería crear nodos option y asignar value y.textContent: evita reinterpretar los nombres como HTML si luego llegan de una fuente externa.

Diferencia clave: .textContent presenta datos como texto en el recibo; innerHTML crea etiquetas option en el registro. Antes del recorrido se reemplaza con =; dentro del recorrido se agrega con +=.

Los saltos de línea del fragmento solo facilitan la lectura. Las llaves cierran los callbacks y el if; los paréntesis cierran las llamadas a forEach y addEventListener.

5. Practica la sustentación

Respuestas cortas, alcance real y demostración

Preguntas rápidas

¿Por qué Number si el input ya es number?

Porque value sigue siendo una cadena. Number la convierte para la comparación; el tipo HTML aporta restricciones del navegador.

¿Qué diferencia hay entre preventDefault y return?

El primero cancela la acción de envío; el segundo sale del callback. En tu código se usan juntos para detener el primer error encontrado.

¿Qué diferencia hay entre push y join?

push modifica el arreglo al añadir un elemento. join produce una cadena con separadores sin modificar el arreglo ni sumar cantidades.

¿Por qué [select_v] y no .select_v?

Los corchetes evalúan la variable y usan su contenido como clave. El punto buscaría la propiedad con ese nombre literal.

¿Quién mueve los datos al recibo y quién guarda las cuentas?

El form mueve los datos mediante GET; JavaScript luego lee la consulta. Ningún archivo de estos proyectos guarda cuentas en una base de datos.

Describe el alcance real del portal

index.html y registro.html usan GET implícito porque no declaran method. required exige completar campos, pero no autentica usuarios ni compara las dos contraseñas. El único script del portal actualiza ubicación. pagar.html no carga JavaScript y sus ocho enlaces Pagar apuntan a esa misma página. Las animaciones hover y los cambios visuales vienen del CSS.

Explicación de apertura que puedes adaptar

En pizzas localizo el formulario, sus casillas y cantidades. Escucho submit y verifico que haya una variedad marcada y que sus cantidades sean al menos uno. Si falla una regla, cancelo el envío, muestro el aviso y termino la función. Si todo pasa, el formulario navega por GET al recibo. Allí leo los parámetros con URLSearchParams, muestro el cliente con textContent, agrego las variedades a un arreglo y las uno con join. En el registro tengo un objeto local por país. Al cambiar el país, limpio los selectores, consulto la clave elegida y recorro sus listas con forEach para crear opciones. El portal actual presenta ese flujo en el navegador; todavía no autentica ni procesa pagos reales.

Ensayo de cinco minutos

Primero intenta enviar pizzas sin marcar ninguna; después marca una y deja la cantidad vacía. Corrige y envía dos variedades: señala sus parámetros en la URL y su texto en el recibo. Finalmente cambia de Colombia a México en el registro y explica la limpieza y los dos recorridos.

Para cualquier línea, responde: qué dato recibe, de qué tipo es, cuándo se ejecuta y qué cambia en pantalla. Basado en el código original y en los comportamientos comprobados en navegador.